Parallel and Distributed Computing

Assignment 6 - Introduction to CUDA

PART A

Device Query

Ques :

Write the device query code, compile and run it on your system. Query enough information to know all the details of your device like GPU name, compute capabilities, max block and grid dimensions, memory sizes, warp size etc.

Method

We use `cudaGetDeviceProperties()` to query all the GPU details. The code queries the device count first, then loops through each device and prints all the properties.

Question A Terminal Output

```
./questionA/device_query
Number of CUDA devices: 1

========== Device 0: NVIDIA GeForce GTX 1650 ==========

--- Compute Capability ---
Compute Capability       : 7.5

--- Processor Info ---
Number of SMs            : 14
Max threads per SM       : 1024
Max threads per block    : 1024
Warp size                : 32

--- Max Block Dimensions ---
Max block dim x          : 1024
Max block dim y          : 1024
Max block dim z          : 64

--- Max Grid Dimensions ---
Max grid dim x           : 2147483647
Max grid dim y           : 65535
Max grid dim z           : 65535

--- Memory Info ---
Global memory            : 4095 MB
Shared memory per block  : 48 KB
Constant memory          : 64 KB
Registers per block      : 65536

--- Other Details ---
Clock rate               : 1.51 GHz
Memory clock rate        : 6.00 GHz
Memory bus width         : 128 bits
L2 cache size            : 1024 KB
Max threads per SM       : 1024
Concurrent kernels       : Yes
ECC enabled              : No
Double precision         : Supported
```

Figure A: Device Query Output

Answers to Questions

1. What is the architecture and compute capability of your GPU?

The compute capability of a GPU tells us what generation of NVIDIA architecture it belongs to and what features it supports. A higher number means newer architecture. For example compute capability 7.5 means Turing, 8.6 means Ampere, 8.9 means Ada Lovelace etc.

My GPU: NVIDIA GeForce GTX 1650 (Turing architecture, compute capability 7.5)

2. What are the maximum block dimensions for your GPU?

Block dimensions tell us the max number of threads we can have in each direction (x, y, z) of a thread block. But the total threads per block cant exceed the max threads per block limit which is usually 1024 on modern GPUs.

From our device query:

Dimension	**Max Value**
Block dim x	1024
Block dim y	1024
Block dim z	64
Max threads/block	1024

3. If max grid dim is 65535 and max block dim is 512, what is the max number of threads?

For a 1D launch:

max threads = max grid dimension $\times$ max block dimension

$= 65535 \times 512$

= **33,553,920 threads**

So roughly 33.5 million threads can be launched in a 1D grid+block configuration.

4. Under what conditions might a programmer choose not to launch the max number of threads?

- if the problem size is small - like if you only have 1000 elements to process then launching 33 million threads is pointless. most threads would sit idle doing nothing
- if each thread needs a lot of registers or shared memory. launching too many threads means each one gets less resources and the GPU might not have enough to actually run them
- if threads need to synchronize a lot. more threads means more synchronization overhead which can slow things down
- sometimes using fewer threads where each thread does more work (thread coarsening) gives better performance because of less launch overhead and better cache usage

5. What can limit a program from launching the max number of threads?

- **register pressure** - GPU has a fixed number of registers per SM. if each thread uses too many registers, fewer threads can be active at the same time
- **shared memory usage** - if a block uses too much shared memory then fewer blocks can fit on an SM simultaneously
- **problem size** - no point launching more threads than data elements you have
- **max threads per block** - even if the grid can be huge, each block has a hard limit (usually 1024)

- **hardware limits** - number of SMs, max warps per SM, max blocks per SM all put a cap on occupancy

6. What is shared memory? How much shared memory is on your GPU?

Shared memory is a small but very fast on-chip memory that is shared between all threads in a block. Think of it like a user-managed L1 cache. Its way faster than global memory (roughly 100x faster) because its physically on the chip right next to the processing cores.

Threads in the same block can use shared memory to share data with each other and communicate. Its really useful for things like parallel reduction (like our array sum in part B) where threads need to combine their results.

Shared memory per block on my GPU: 48 KB

7. What is global memory? How much global memory is on your GPU?

Global memory is the main memory of the GPU - the big VRAM you see on the spec sheet. Its the largest memory available but also the slowest. All threads from all blocks can read and write to it.

When we do `cudaMalloc`, the memory gets allocated in global memory. Data has to be copied from CPU RAM to GPU global memory using `cudaMemcpy` before the GPU can work on it.

Global memory on my GPU: 4096 MB

8. What is constant memory? How much constant memory is on your GPU?

Constant memory is a special read-only memory on the GPU. Its cached so reads are fast (as fast as registers if all threads in a warp read the same address). But you cant write to it from a kernel - you set the values from the CPU side before launching the kernel.

Good for things like physical constants, lookup tables, kernel parameters that all threads

need to read but nobody changes.

Constant memory is always **64 KB** on all CUDA GPUs - thats a fixed hardware limit.

9. What does warp size signify? What is your GPU's warp size?

Warp size is the number of threads that execute together in lockstep on the GPU. The GPU doesnt execute threads one at a time - it executes a whole warp (group of threads) together. All threads in a warp run the same instruction at the same time. This is called SIMT (Single Instruction Multiple Threads).

If threads in a warp take different paths in an if-else statement, the warp has to execute both paths one after another. This is called **warp divergence** and its bad for performance because half the threads sit idle during each path.

Warp size on all NVIDIA GPUs: **32 threads**

10. Is double precision supported on your GPU?

Double precision (64-bit float) is supported on GPUs with compute capability 1.3 and above. So basically all modern GPUs support it. But the performance is usually much worse than single precision:

- on consumer GPUs like GeForce - double precision runs at 1/32 the speed of single precision
- on workstation GPUs like Tesla/A100 - its much better, about 1/2 the speed

Yes, double precision is supported on this GPU.

PART B

Array Sum using CUDA

Ques :

Write a CUDA program to calculate the sum of elements in a single precision float array. Follow these steps: allocate device memory, copy host to device, set up block/grid dimensions, invoke kernel, copy results back, free device memory, and write the reduction kernel.

Approach

We use parallel reduction in shared memory. The idea is simple - each block loads its chunk of the array into shared memory, then we do a tree-based reduction where we keep halving the active threads and adding pairs together. At the end, thread 0 of each block has the sum of that blocks elements. Then we copy all the block sums back to CPU and add them up.

Step by step:

1. **Allocate device memory** - `cudaMalloc` for the input array and an array to hold each blocks partial sum
2. **Copy host memory to device** - `cudaMemcpy` with `cudaMemcpyHostToDevice` copies our input array from CPU RAM to GPU global memory
3. **Initialize thread block and kernel grid dimensions** - we use 256 threads per block. grid size = ceil(N / blockSize) so every element gets exactly one thread
4. **Invoke CUDA kernel** - launch with shared memory size = blockSize * sizeof(float). the kernel does tree reduction inside each block

5. **Copy results from device to host** - `cudaMemcpy` with `cudaMemcpyDeviceToHost` copies the block sums back to CPU

6. **Free device memory** - `cudaFree` releases the GPU memory we allocated

7. **The kernel** - uses `extern __shared__ float sdata[]` for fast reduction within each block

```
Question B Terminal Output

./questionB/array_sum

Array Sum using CUDA

Array size: 1048576 elements


CPU sum (for verification): 1048576.00

Block size: 256

Grid size : 4096


GPU sum: 1048576.00

Kernel time: 4.9230 ms

Difference: 0.000000e+00

PASSED - results match
```

Figure B: Array Sum Output

PART C

Matrix Addition using CUDA

Ques :

Perform matrix addition of two large integer matrices in CUDA. Answer questions about floating point operations, global memory reads, and global memory writes.

Approach

Matrix addition is embarrassingly parallel - each element is independent. We use a 2D grid and 2D blocks to map threads directly to matrix elements. Each thread computes C[i][j] = A[i][j] + B[i][j].

We use 16x16 thread blocks (256 threads total per block) and calculate the grid dimensions to cover the entire matrix.

```
Question C Terminal Output

./questionC/matrix_add
Matrix Addition using CUDA
Matrix size: 4096 x 4096

Block dimensions: 16 x 16
Grid dimensions : 256 x 256

PASSED - all elements correct
Kernel time: 4.0536 ms

--- Analysis ---
Total elements (N x N)          : 16777216
Floating point ops (additions)  : 16777216
Global memory reads             : 33554432 (N from A + N from B = 2N)
Global memory writes            : 16777216 (N to C)
```

Figure C: Matrix Addition Output

Answers to Questions

For a matrix of size N $\times$ N where N = 4096:

1. How many floating point operations are being performed in the matrix addition kernel?

Each element needs exactly 1 addition operation.

Total operations = N $\times$ N = 4096 $\times$ 4096 = **16,777,216 operations**

(note: technically these are integer additions since we use int matrices but the concept is the same)

2. How many global memory reads are being performed by your kernel?

For each element we read:

- 1 element from matrix A (global memory read)
- 1 element from matrix B (global memory read)

Total reads = 2 × N × N = 2 × 4096 × 4096 = **33,554,432 reads**

3. How many global memory writes are being performed by your kernel?

For each element we write:

- 1 element to matrix C (global memory write)

Total writes = N × N = 4096 × 4096 = **16,777,216 writes**

Metric	**Formula**	**Value (N=4096)**
Arithmetic operations	N × N	16,777,216
Global memory reads	2 × N × N	33,554,432
Global memory writes	N × N	16,777,216
Total memory accesses	3 × N × N	50,331,648
Arithmetic intensity	ops / mem accesses	1/3

Table 1: Memory access analysis for matrix addition

The arithmetic intensity is only 1/3 which is very low. This means matrix addition is a **memory-bound** operation - the GPU spends most of its time moving data around and very little time doing actual computation. Compare this with matrix multiplication where

the arithmetic intensity is O(N) which makes it compute-bound and much better suited for GPUs.

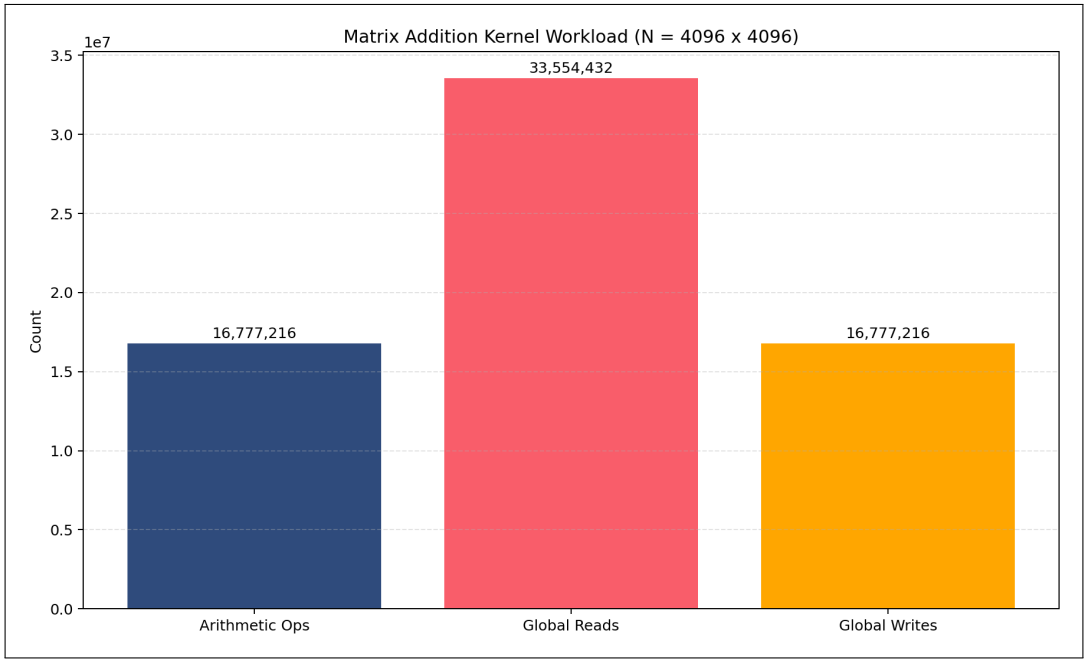

Figure D: Matrix Addition Workload Graph

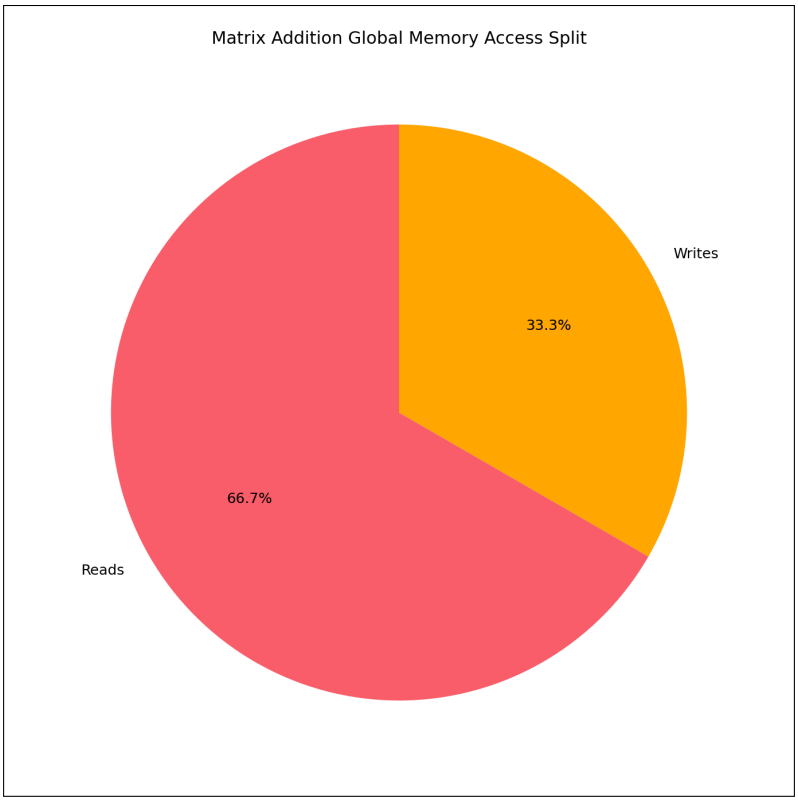

Figure E: Matrix Addition Memory Access Split

Observations

1. **CUDA programming follows a standard pattern** - allocate on GPU, copy data over, launch kernel, copy results back, free GPU memory. this host-device data transfer is often the bottleneck, not the actual computation

2. **Thread block size of 256 works well as a default** - its a multiple of the warp size (32), gives good occupancy on most GPUs, and balances register/shared memory usage. 16x16 blocks work great for 2D problems like matrix operations

3. **Shared memory reduction is much faster than naive approaches** - for the array sum, doing reduction in shared memory avoids millions of slow global memory accesses. the tree reduction pattern (halving active threads each step) does log2(blockSize) steps instead of blockSize steps

4. **Matrix addition is too simple to fully utilize a GPU** - with arithmetic intensity of 1/3, the GPU cores are mostly waiting for memory. This is why matrix multiplication (which reuses data heavily) shows much better GPU speedups

5. **Boundary checking is important** - when the array or matrix size doesnt divide evenly by the block size, some threads will be out of bounds. The `if (i < n)` check prevents illegal memory accesses

Inferences

1. **GPUs are best for compute-heavy parallel problems** - matrix addition doesnt benefit much from GPU because its memory-bound. but problems like matrix multiplication, deep learning, simulations etc where each data element needs lots of computation are where GPUs really shine

2. **Memory hierarchy matters a lot** - just like CPU caches, using shared memory

instead of global memory can give massive speedups. our reduction kernel would be way slower if we did everything in global memory

3. **Warp-level thinking is important** - since 32 threads execute together, things like warp divergence (branching within a warp) and memory coalescing (adjacent threads accessing adjacent memory) directly impact performance

4. **The host-device transfer overhead** - for small problems, the time spent copying data between CPU and GPU can be more than the computation itself. CUDA is worth it only when the computation is heavy enough to offset this transfer cost

5. **Occupancy isnt everything** - launching more threads doesnt always mean better performance. sometimes fewer threads with more work per thread (thread coarsening) gives better results because of reduced overhead and better cache utilization

Submitted to : Dr Saif Nalband

Submitted by : Arpan Goyal

Roll No. : 102303479